DETECTOR DE OBJETOS

IXTLI



LOS ESO BRAD
ASIEL GUEVARA
ALBERTO DIAZ
ISRAEL GALLEGOS
CINTHYA BALTAZAR

DESCRIPCIÓN

Este proyecto se desarrolló en crear una Inteligencia artificial para detectar una variedad de objetos atreves de una de una cámara pasa un objeto, la IA detecta la forma y colores de objetos.

Nuestra IA esta desarrollada con diferentes herramientas las cuales son usadas en el mundo del desarrollo una de ellas YOLOv11n una herramienta de IA pre-entrenada en donde contiene de forma predeterminada ciertas configuraciones para su uso y deteccion de objetos, una de la herramientas utilizadas es ROBOFLOW, el cual se utilizo para simplificar la creación de modelos de visión en las computadoras utilizadas, usando herramientas de uso manual y automatico para la etiquetación de imgenes con poligonos de segmentacion. Gracias a estas herramientas pudimos crear un modelo propio entrenado especificamente para detectar los objetos solicitados por el jurado de hackaton.

Sistema de detección de objetos en tiempo real con la cámara del equipo, basado en YOLO11n (Ultralytics) con ajuste fino sobre un dataset propio.

Clases detectadas: Botella, Celular, Desarmador, Taza, Tijeras.

1. Requisitos

- Python 3.10 o superior
- Cámara web
- (Opcional) GPU con CUDA para acelerar el entrenamiento

2. Instalación

git clone https://github.com/arielit3/Mechatronics

cd Mechatronics

python -m venv .venv

Windows:

.venv\Scripts\activate

Linux / Mac:

source .venv/bin/activate

pip install -r requirements.txt

Si installer.py instala las dependencias automáticamente, añade aquí: "Alternativa: ejecutar python installer.py".

3. Ejecución

Detección en vivo con la cámara:

python Deteccion/detection.py

Evaluación del modelo sobre el conjunto de prueba (precision, recall, mAP y matriz de confusión):

python metricas/evaluar_modelo.py

Los resultados se guardan en metricas/resultados/.

4. Estructura del proyecto

Mechatronics/

```
├── dataset/              # Imágenes etiquetadas, una carpeta por clase
│   ├── Botella/
│   ├── Celular/
│   ├── Desarmador/
│   ├── Taza/
│   └── Tijeras/
├── Deteccion/
│   ├── best.pt           # Pesos del modelo entrenado
│   └── detection.py      # Detección en vivo con la cámara
├── metricas/
│   ├── evaluar_modelo.py # Script de evaluación reproducible
│   └── resultados/       # Métricas y matriz de confusión
├── runs/                 # Salidas de entrenamiento (Ultralytics)
├── src/                  # [describe brevemente qué contiene]
├── installer.py          # [describe brevemente su función]
├── IXTLI.bat             # [describe brevemente su función]
├── requirements.txt
├── yolo11n.pt            # Pesos preentrenados de partida
└── README.md
```

5. Dataset

- Captura: [N] imágenes con la cámara del equipo, [20] por clase.
- Variaciones: iluminación (natural / artificial / baja luz), fondo (describe) y posición (describe).
- Etiquetado: realizado con LabelImg / Roboflow / CVAT, formato YOLO.
- División: 70% train / 15% val / 15% test, con semilla fija para que sea reproducible.

6. Modelo y entrenamiento

- Modelo: YOLO11n preentrenado (yolo11n.pt) con ajuste fino sobre nuestras 5 clases. Los pesos resultantes están en Deteccion/best.pt.
- Hiperparámetros principales: épocas = [100], imgsz = [640], batch = [16], semilla = [42].
- Aumento de datos: variación de color/iluminación (HSV), rotación, traslación, escala, volteo horizontal y mosaic.

7. Limitaciones conocidas

- Dataset pequeño (imágenes); el conjunto de prueba tiene pocas muestras por clase, por lo que las métricas pueden ser inestables.
- Baja precisión con poca luz o fondos muy distintos a los del entrenamiento.
- Confusión entre [clase A] y [clase B] (ver matriz de confusión) estas matrices pueden ser botella, taza.
- Rendimiento de X FPS en [CPU / GPU].
- Imágenes capturadas con una sola cámara, por lo que puede no generalizar a otros dispositivos.

8. Mejoras propuestas

- Ampliar el dataset con más variaciones de luz, fondo, distancia y ángulo.
- Capturar con distintas cámaras para mejorar la generalización.
- Probar modelos más grandes (yolo11s / yolo11m) o más épocas de entrenamiento.
- Reforzar con más ejemplos las clases que más se confunden.
- Mejorar la interfaz visual de la cámara (etiquetas, colores y contador de FPS).**u**
- Sistema de alerta al momento de portar unas tijeras.

9. Fuentes citadas

Asim. (2025). *Screwdriver dataset* [Conjunto de datos de código abierto]. Roboflow Universe. https://universe.roboflow.com/asim-wcmjq/screwdriver-dtbch

eco. (s. f.). *Cellphone dataset* [Conjunto de datos de código abierto]. Roboflow Universe. https://universe.roboflow.com/eco-18oum/cellphone-ah9eu

FYP. (s. f.). *Plastic Bottle 2.0* [Conjunto de datos de código abierto]. Roboflow Universe. https://universe.roboflow.com/fyp-li8zz/plastic-bottle-2.0

Kaya, O. (2024). *1750 dataset* (Versión 1) [Conjunto de datos de código abierto]. Roboflow Universe. https://universe.roboflow.com/onur-kaya-yjuml/1750/dataset/1

Singtothong, C. (2023). *Scissors dataset* [Conjunto de datos de código abierto]. Roboflow Universe. https://universe.roboflow.com/chutinan-singtothong-yfgms/scissors-09ewz